Basic (single-agent support bot design) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order against your own written design before reading ahead.

- Hint 1 — What to ask, and the obvious first design
- Hint 2 — The plan, and almost the whole design

Hint 1 — What to ask, and the obvious first design

This is a retrieval-augmented generation (RAG) design — Doc08 material — wrapped in a single tool-calling agent (Doc11's simplest pattern: one agent, one or more tools, no multi-agent routing needed for a task this size). The two real pieces:

- A `search_docs` tool that retrieves over the product documentation (embeddings + vector search, or plain keyword search at very small scale — matching the tool to the actual traffic, not defaulting to the fanciest option available).
- A routing decision after retrieval: is there a real basis for an answer, or not? Three real ways to define "unsure," each with a different reliability:
- **Retrieval-score threshold** — if nothing scores above X similarity, treat it as unsure. Cheap and mechanical, but doesn't understand meaning.
- **LLM self-reported confidence** ("rate your confidence 1-10") — cheap to add, but models aren't reliably calibrated about their own confidence, so trusting this alone is risky.
- **A grounding check** — did the retrieved docs actually cover the specific thing asked, or is the model filling the gap with its own general knowledge? This is the one that actually protects against confidently stating something product-specific that isn't true.

State here is simple: per-conversation history plus the most recent retrieval result, so the agent's answer can be traced back to what it actually found.

What a senior reviewer pushes on:

The naive version of this design uses the LLM's own stated confidence to decide when to hand off — and that's exactly the piece a senior reviewer will push on. A model can sound completely confident while stating something the docs never said, because "sounding confident" and "grounded in retrieved text" are not the same signal. The safer check asks a narrower question: does at least one retrieved chunk actually support this specific claim — and treats an answer that leans on the model's

general knowledge instead of the retrieved text as automatically unsure, no matter how fluent it sounds.

A stakeholder will also ask "why hand off to a human at all instead of just saying 'I don't know'" — the honest answer is that a real support flow needs an actual resolution path, not a dead end. So the design has to name where handoff sends the conversation (a queue? a live agent? a ticket?) and what travels with it — the user's question and what the bot already tried, so the human doesn't start from zero.

Cost/latency: RAG adds one retrieval step before generation. For a single-agent bot at modest traffic that's fine as-is. Caching repeated queries only becomes a justified addition once volume actually requires it — adding it now, for traffic nobody mentioned, is the exact "inspiration instead of matching" mistake Core Concepts warns about.

Hint 2 — The plan, and almost the whole design

```
Agent: single support agent (Doc11's simplest pattern — no multi-agent
    routing needed for one clear task)
Tool: search_docs(query) -> ranked chunks from the product documentation
State: conversation history + last retrieval result, per session

Routing:
    call search_docs(user question)
    if best chunk score < threshold:
        route to human handoff
    else:
        generate an answer grounded in the retrieved chunk(s)

Human handoff:
    package: the user's question + what was retrieved (if anything) +
    why it was flagged unsure
    send to: the support queue (exact destination depends on what the
    business already uses — ticketing, live-chat escalation, etc.)
```

Hardened for a senior review:

```
Agent: single support agent
Tools:
    search_docs(query) -> ranked chunks + scores
    escalate_to_human(conversation, reason) -> ticket id

Grounding check (replaces naive self-confidence):
    retrieve chunks
    if no chunk clears the similarity threshold: UNSURE -> escalate
```

```
    else: generate the answer, but constrain the prompt to "answer only
    using the text below, say you don't know if it isn't there" — this
    catches the case where retrieval found something loosely related
    but not actually on-point

Failure handling (Doc14 vocabulary):
    search_docs times out / errors -> don't crash, escalate with reason
    "docs search unavailable"
    LLM call fails -> retry once, then escalate rather than retrying
    forever

Monitoring:
    log every escalation with its reason (score-based miss vs.
    grounding-based miss) — this becomes a signal about gaps in the
    docs themselves, not just a bot-quality metric

Scaling note (only if traffic actually requires it):
    cache repeated queries' retrieval results
```

Full solution: Show me the solution